THREAD MANAGEMENT

2026 SPRING

JINHO LEE

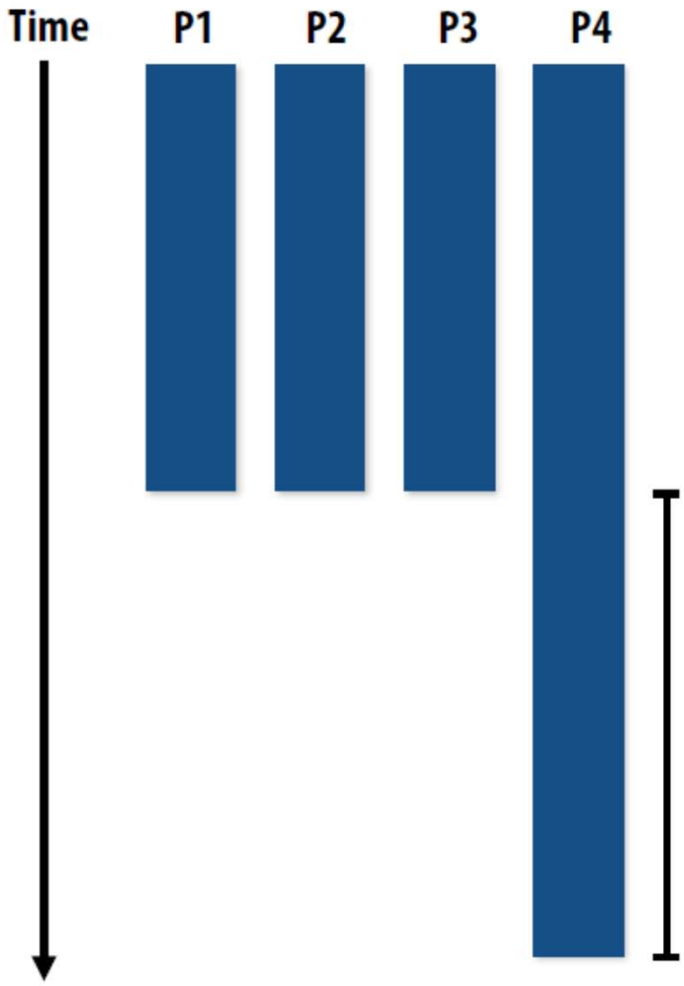
ECE, SNU

CONTENT

- Workload balancing
  - Static assignment
  - Dynamic assignment
- Thread pooling
  - Workqueue
  - Thread-local workqueue

WORKLOAD BALANCING

- Obviously, balancing your workload is critical to the performance



Small amount of load imbalance can significantly degrade speedup

- P4 does 2x more work

→ P4 takes 2x longer to complete

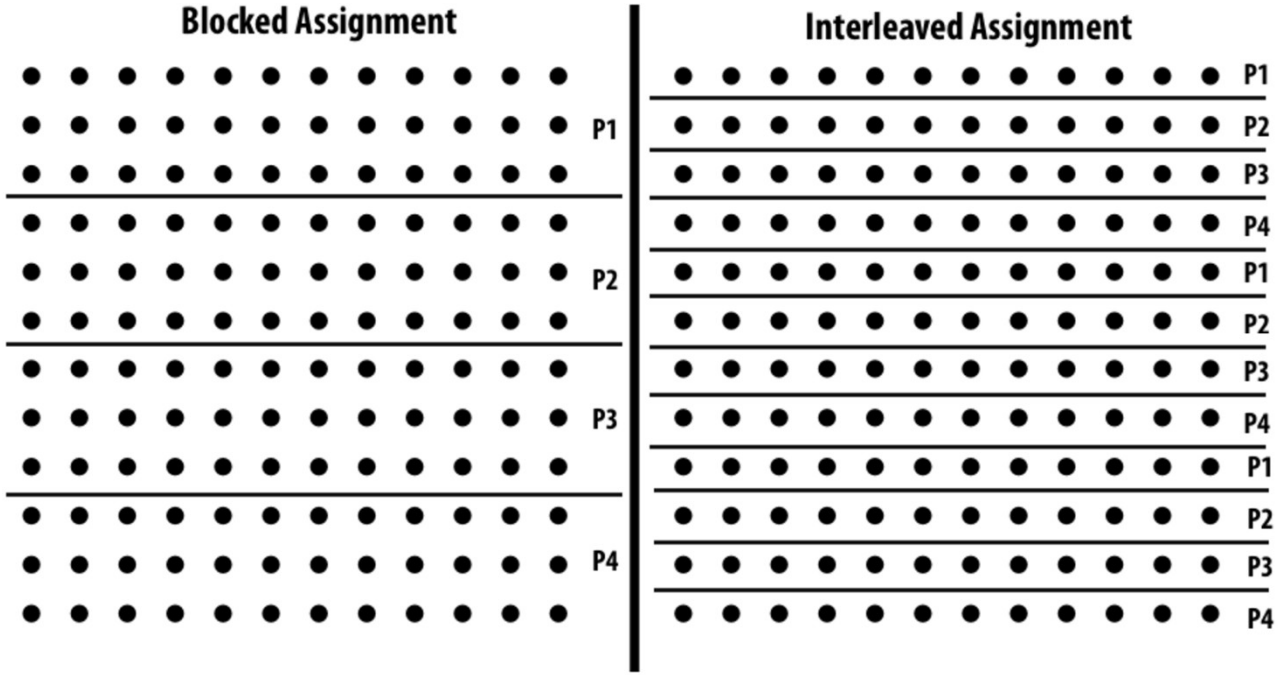
→ 50% of parallel program's runtime is serial execution

- This portion looks like a lot, but only 20% of the total workload

- Then, How do you balance?

STATIC ASSIGNMENT

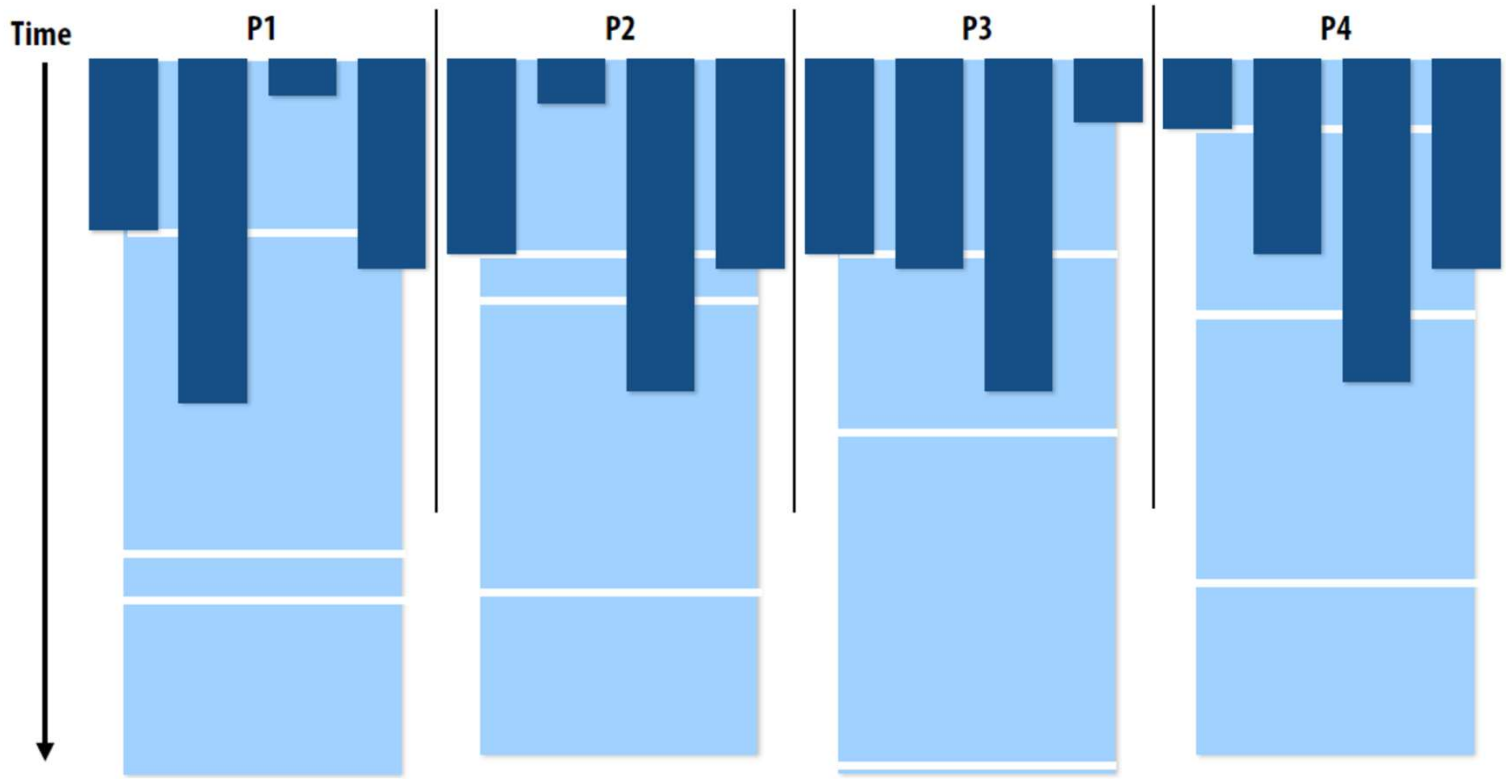
- Assume we're calculating sum(A) on a matrix A
  - Basically a reduce problem
- <u>Amount of work is equal per item</u> (matrix element)
- Giving equal amount of items to each thread is enough



- Simple, no overhead

STATIC ASSIGNMENT

- Assume we're calculating sum(A) on a matrix A
  - Basically a reduce problem
- Amount of work is equal per item (matrix element)
- The amount of work has to be predictable (reduce is a good example)
  - They may not be equal, but predictability is enough!



DYNAMIC ASSIGNMENT

- You have a pile of work, each worker comes and gets one by one
  - Assume you're working on input[N]
- Example:

```
#include <iostream>
#include <thread>
#include <vector>

void worker(int* input, int start, int size, int*
output)
{
     while(i < N) {
          global.lock();
          i = counter++;
          global.unlock();
          do_something_on(input[i]);
     }
}
```

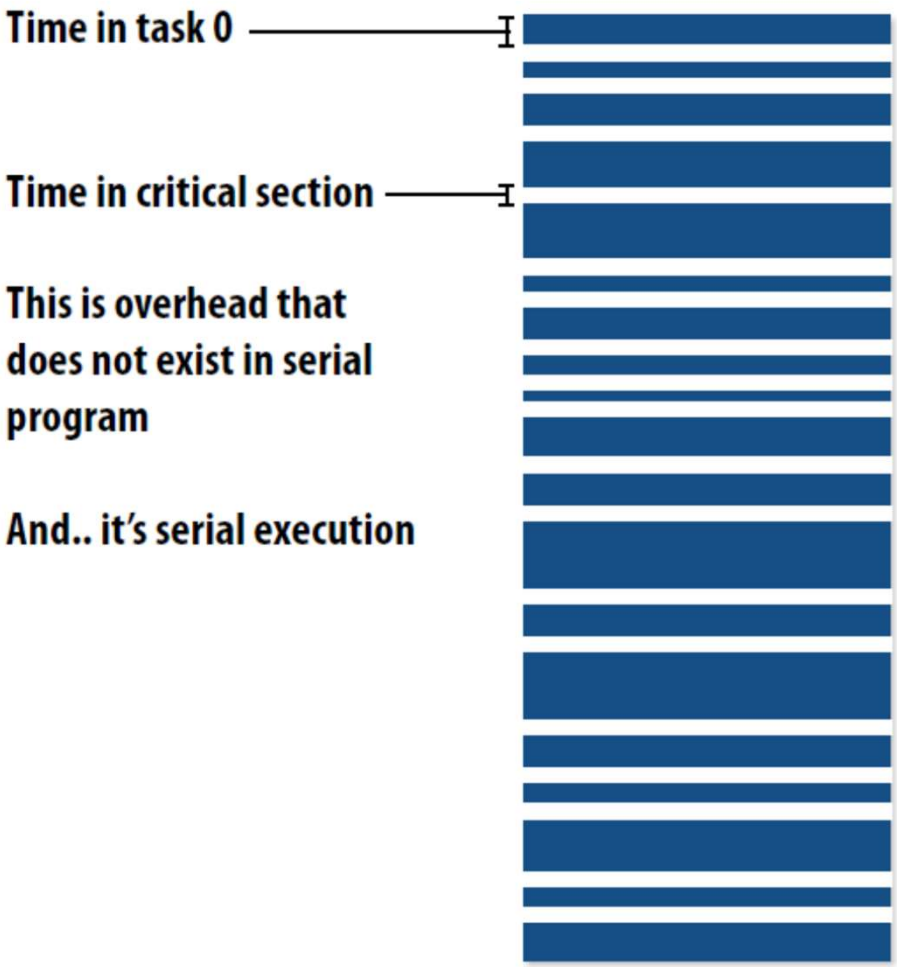
- An example way of distributing work by single elements
- Of course you can increase the counter by 10, 100, 1000, etc

DYNAMIC ASSIGNMENT

- What should be the dynamic assignment granularity?

```
#include <iostream>
#include <thread>
#include <vector>

void worker(int* input, int start, int
size, int* output)
{
     while(i < N) {
          global.lock();
          i = counter++;
          global.unlock();
          do_something_on(input[i]);
     }
}
```



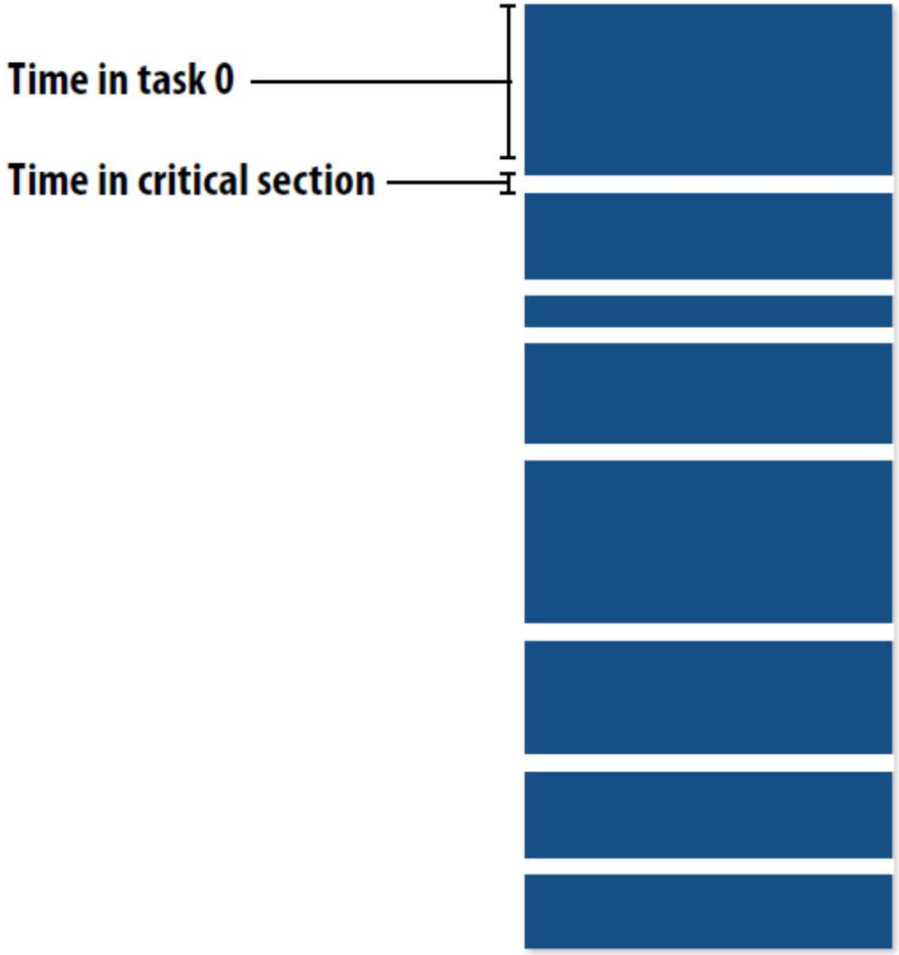
- (+) Good balance
  - Maximum imbalance = size of a chun
- (−) With small granularity, you spend too much overhead on the job dispatching

DYNAMIC ASSIGNMENT

- What should be the dynamic assignment

```
#include <iostream>
#include <thread>
#include <vector>

void worker(int* input, int start, int
size, int* output)
{
     while(i < N) {
          global.lock();
          i = counter;
          counter += 1000;
          global.unlock();
          //worksize=1000
          do_something_on(input[i], 1000);
     }
}
```

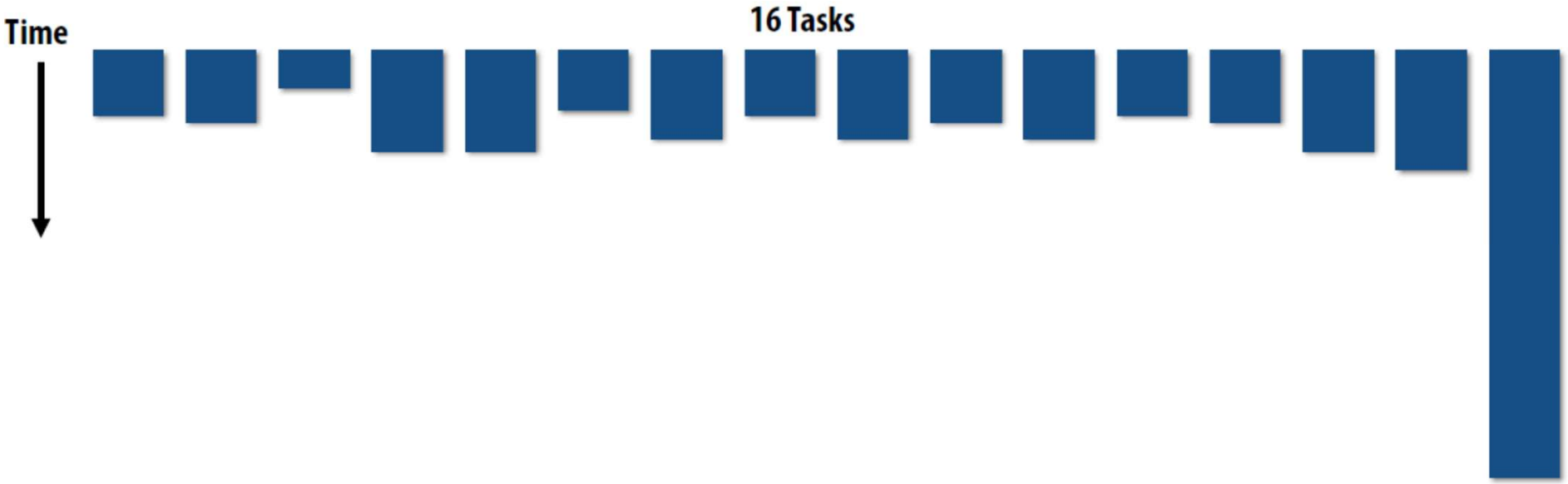


- (+) Less overhead inside critical sectic
- (-) worse balance
    - Maximum imbalance = size of a chunk

GENERAL RULES FOR THE GRANULARITY

- We want many more tasks than the number of threads (or processors)
  - e.g., 17 equal-sized tasks, with 10 threads: destined to suffer from imbalance!
- But not too many
  - Critical section is expensive!
  - Context switch overhead
- Ideal granularity?
  - That depends….
- <u>Practical best answer: try and measure!</u>

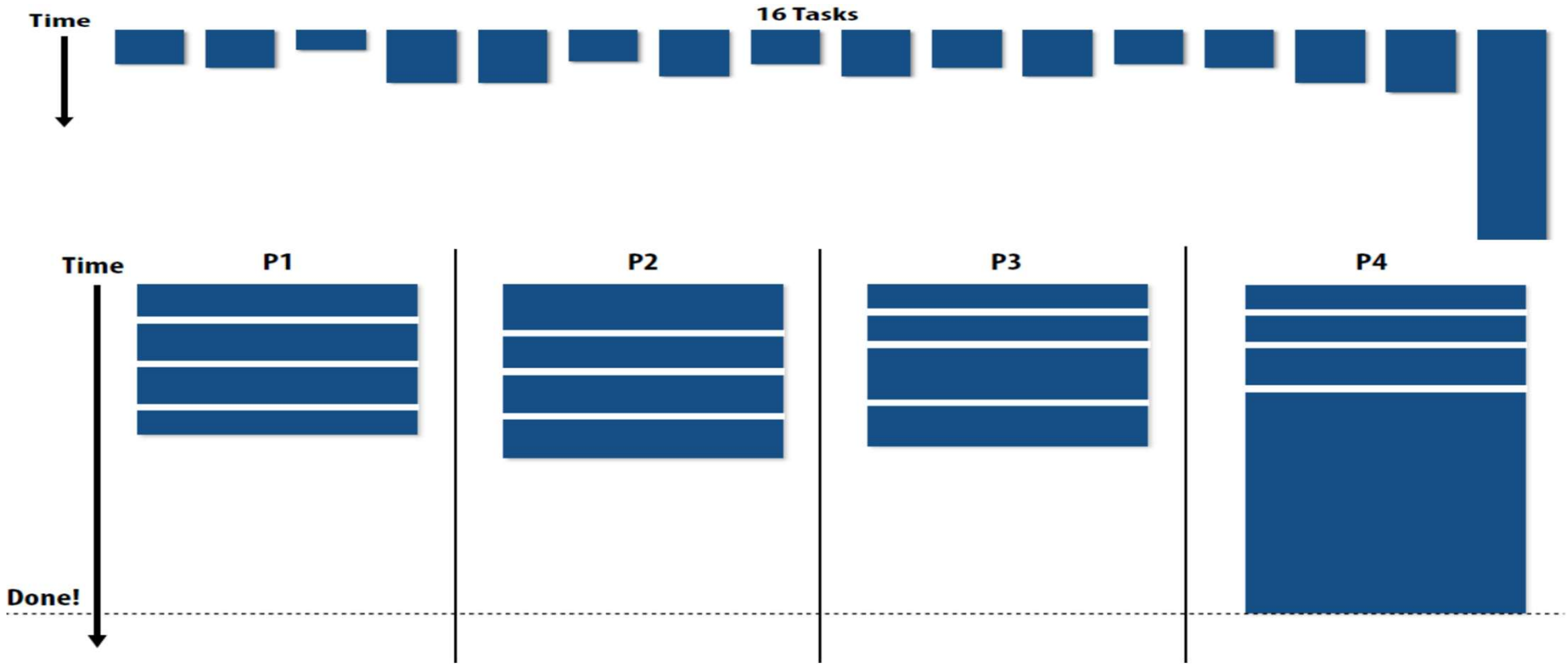
HOW TO DYNAMICALLY SCHEDULE



- Assume you have 4 threads
- How would you schedule?

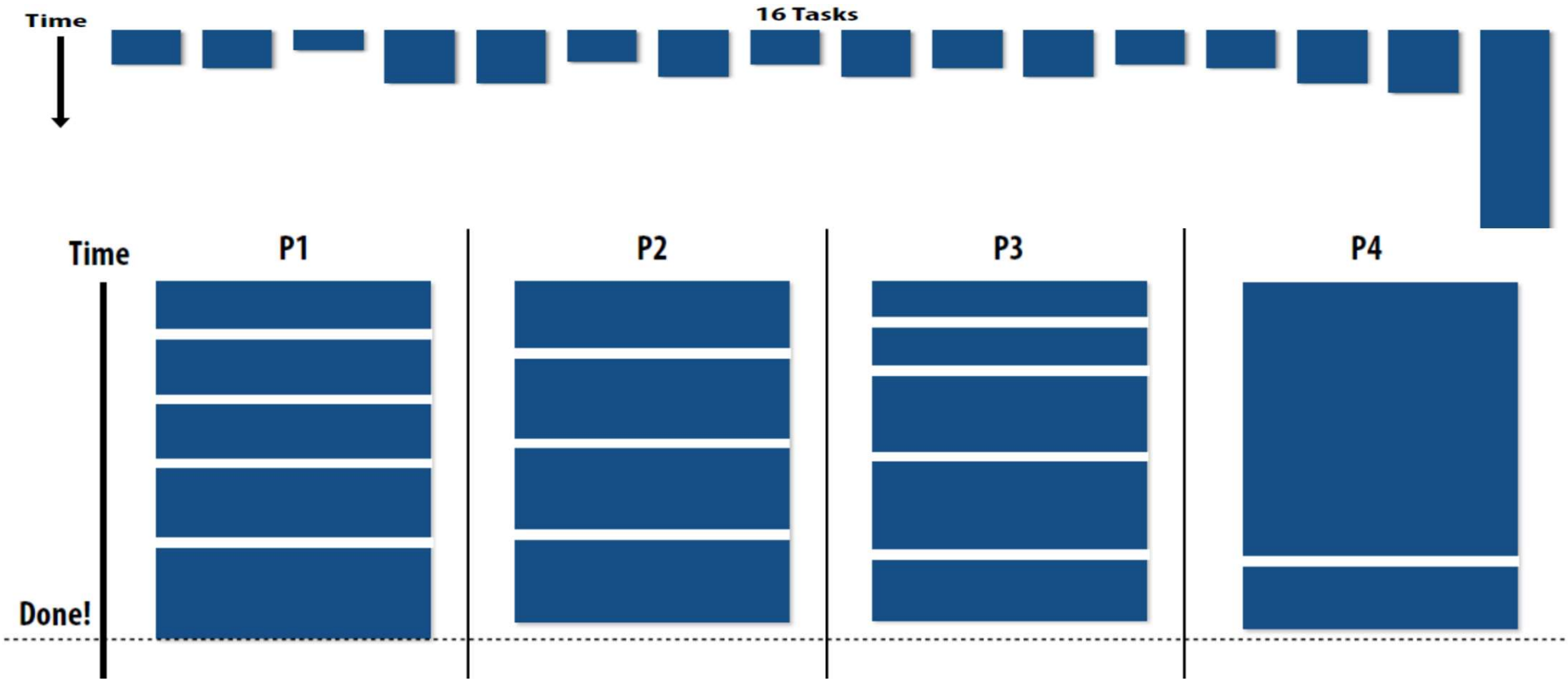
- Do you really wanna find THE OPTIMAL solution??

HOW TO DYNAMICALLY SCHEDULE



- "From the left" greedy solution
- Can suffer from a large work being scheduled near the end
- Divide it further?
  - May not be possible
  - Too small granularity?

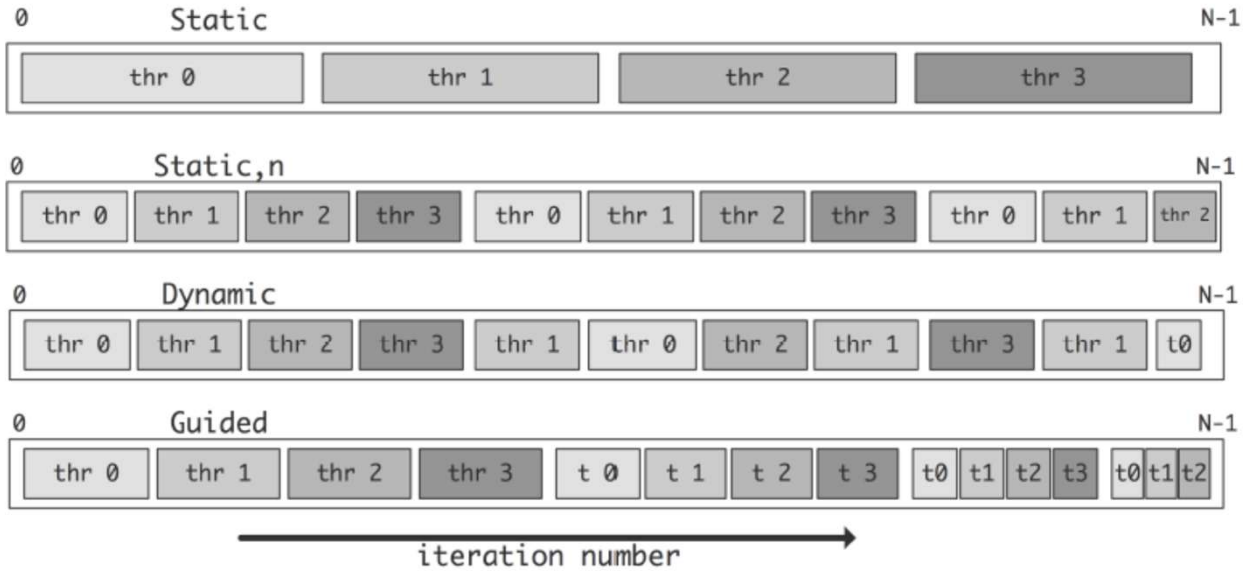
HOW TO DYNAMICALLY SCHEDULE



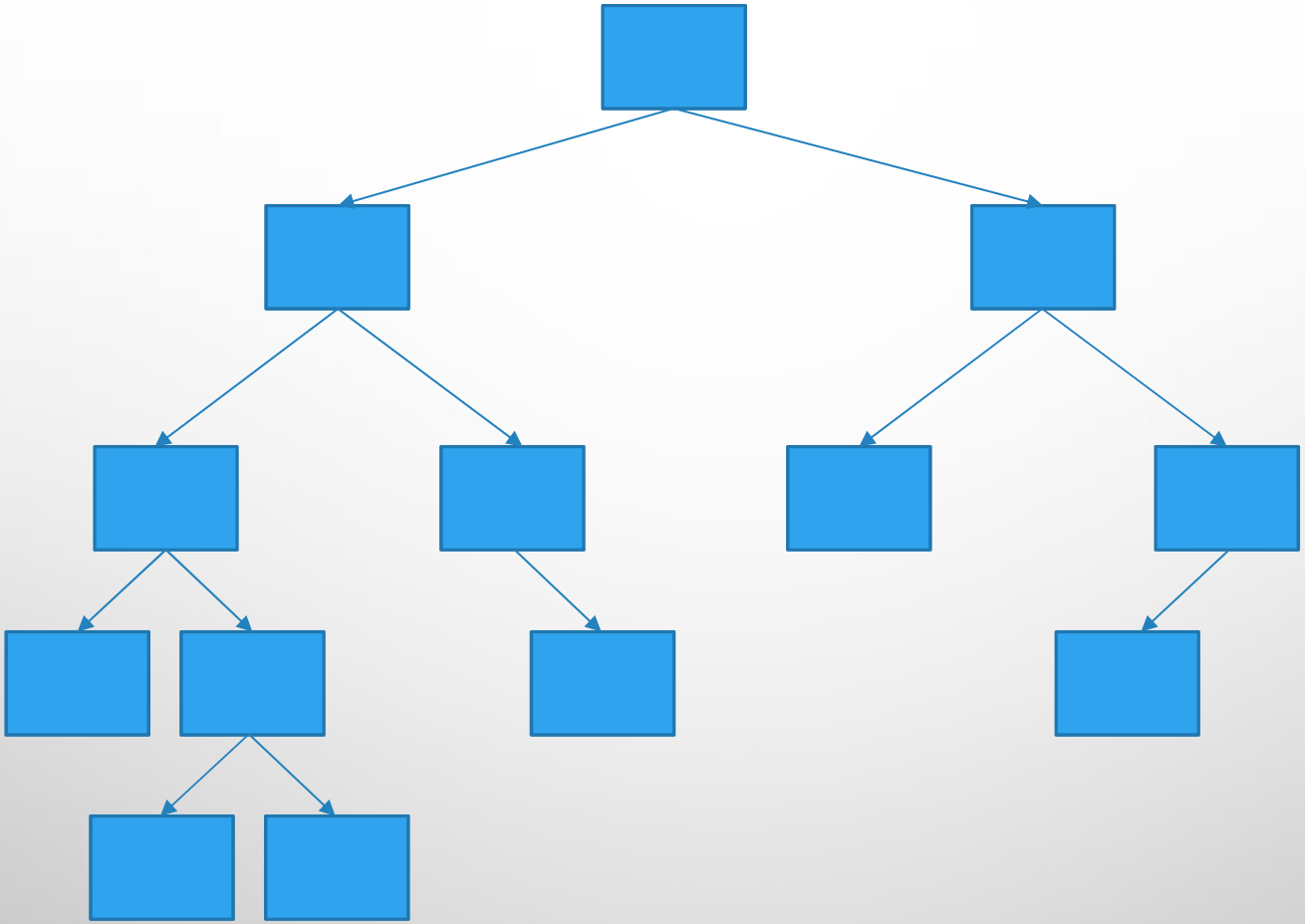
- "largest first" greedy solution
- Likely to do better
- If you can figure out which is larger beforehands…..
  - Sorting might be needed

SCHEDULING WORKS

- Static: iterations are divided into blocks of block_size, and statically assigned to threads
- Dynamic: when a thread finishes a block, it is assigned the next block
  - Useful when work per iteration is irregular
- Guided: a dynamic scheduling, but the block size decreases over time
  - Useful when work per iteration is relatively regular, but threads arrives at varying times
  - Beginning: minimize switching overhead,
  - End: minimize imbalance

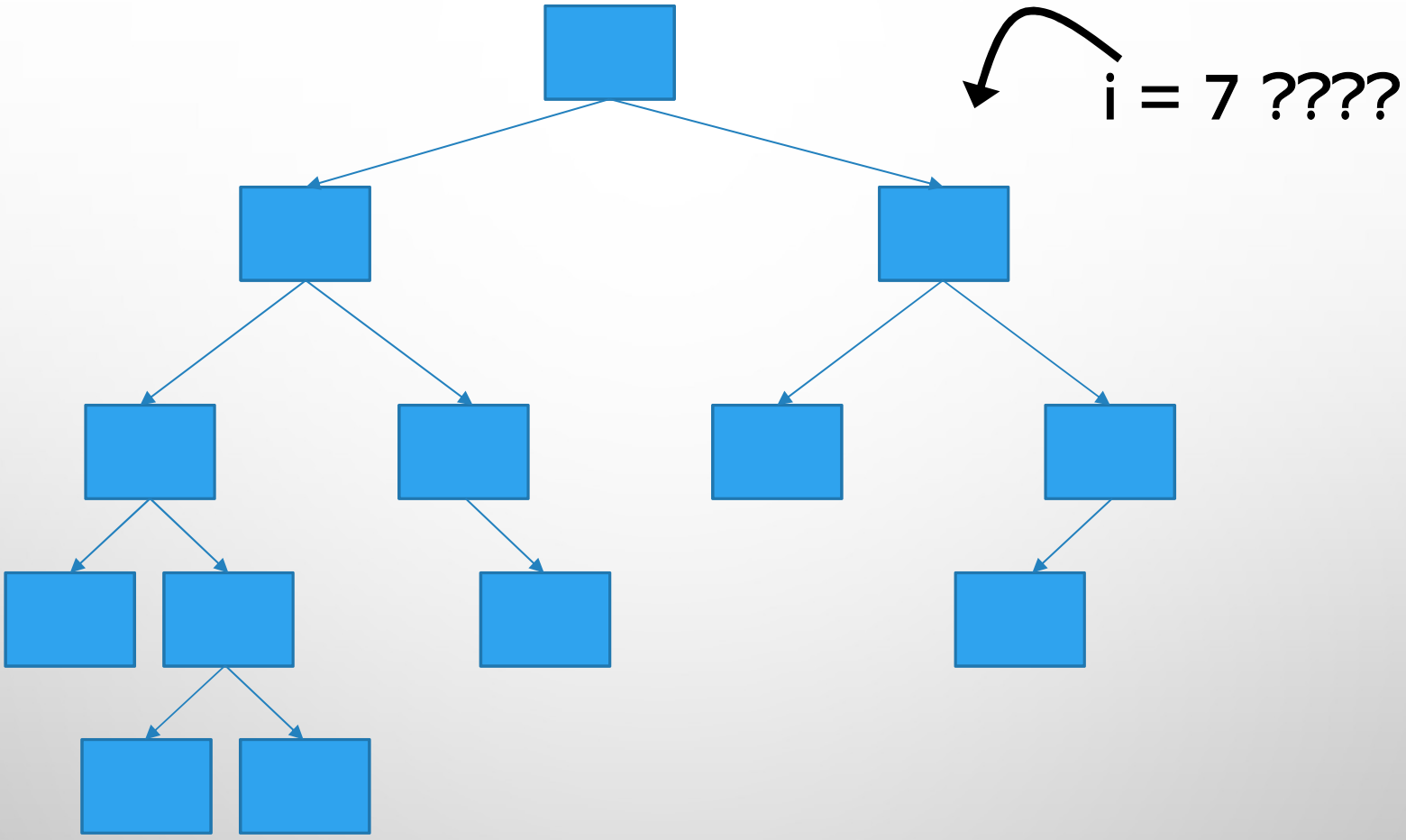


THREAD POOL (WORK QUEUE) - MOTIVATION



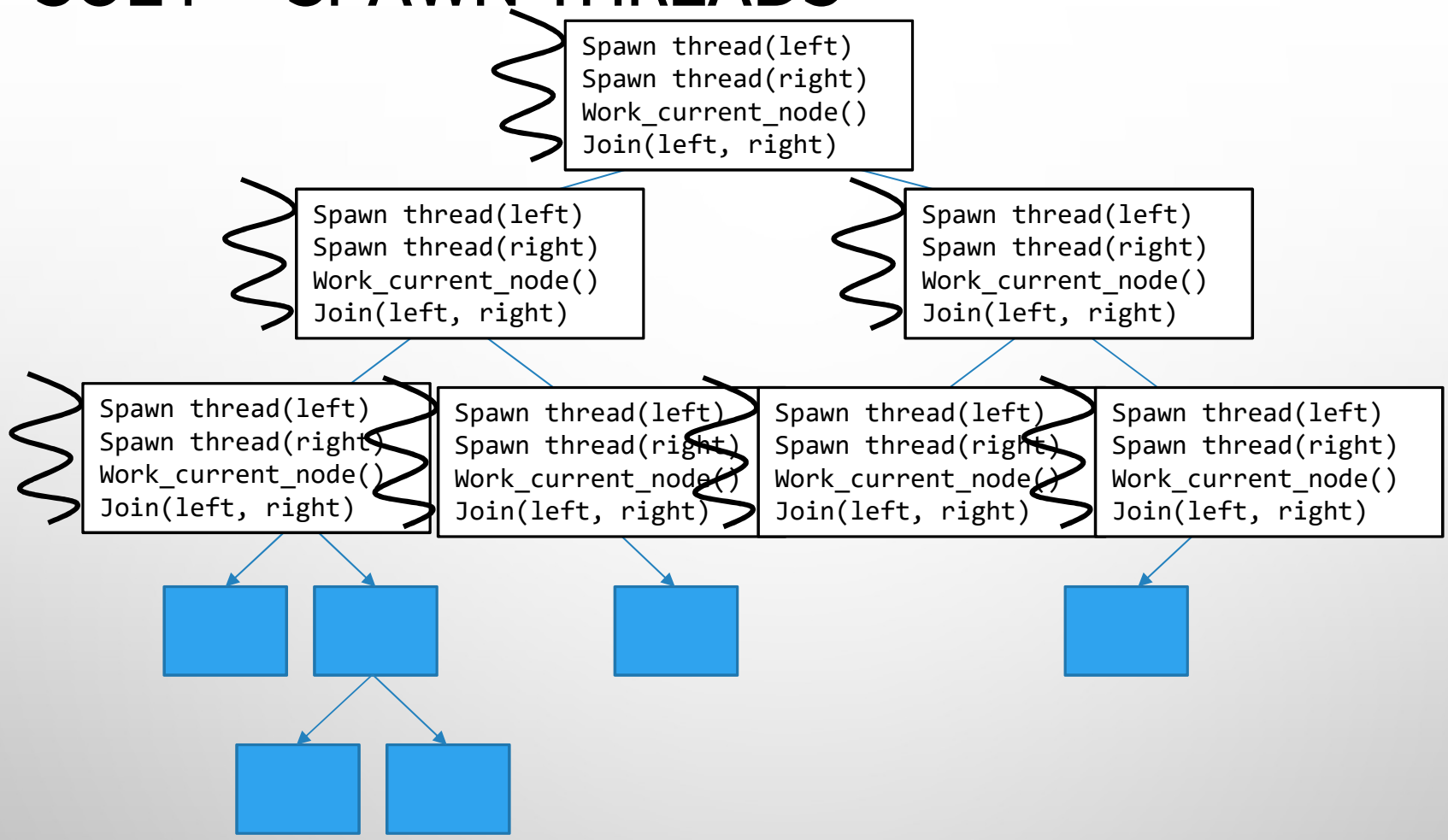
- Assume you want to traverse a tree from the root
- There's some work assigned to each node
- What do you do?

THREAD POOL (WORK QUEUE) - MOTIVATION



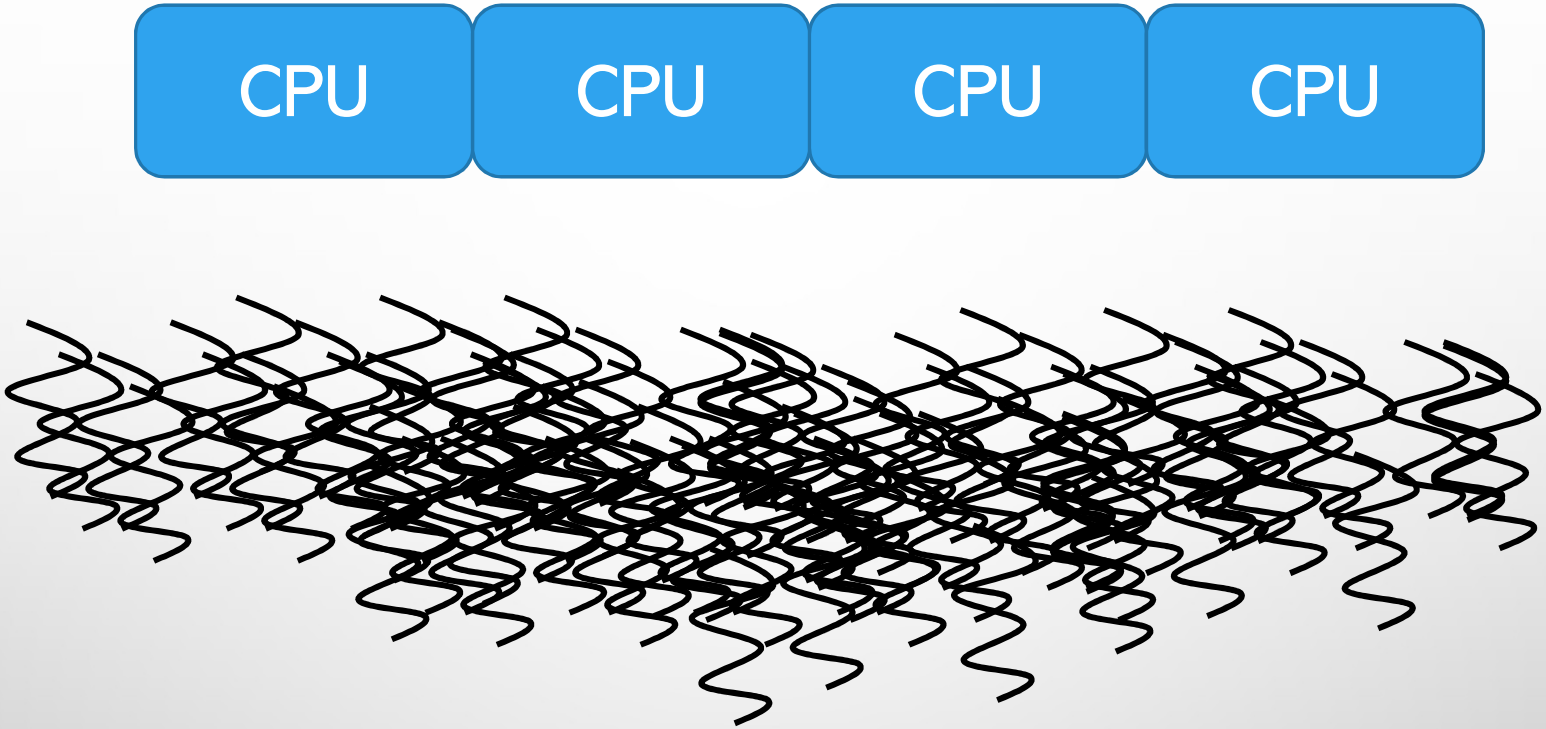
- Static assignment will fail – how do you split in the first place?
- Dynamic assignment is likely.. But <u>counter does not work</u>

SOL1 – SPAWN THREADS



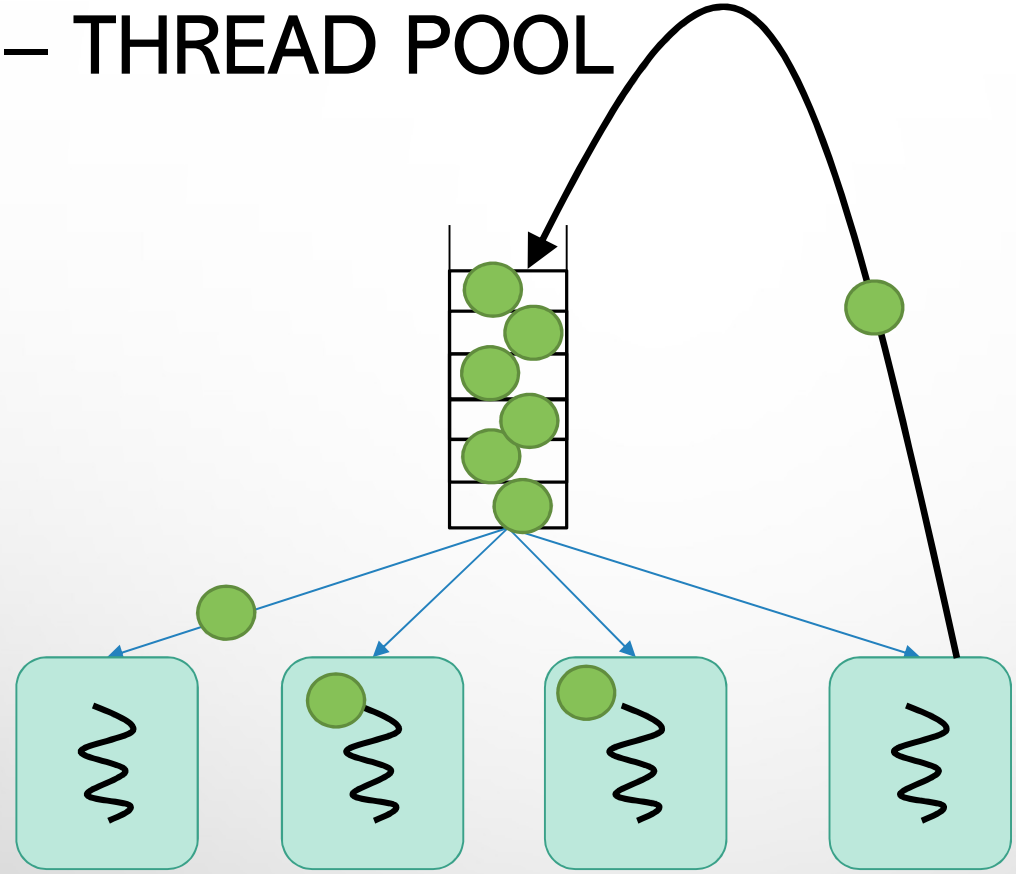
- Spawn thread for children…
- But how many threads??

SOL1 – SPAWN THREADS



- Having more threads than the number of cores sometimes helps
- With too much, you will suffer from <u>**context switching**</u> hell
  - Few hundreds of microseconds to a few milliseconds for a context switch
- Having a fixed number of thread helps

SOL2 – THREAD POOL



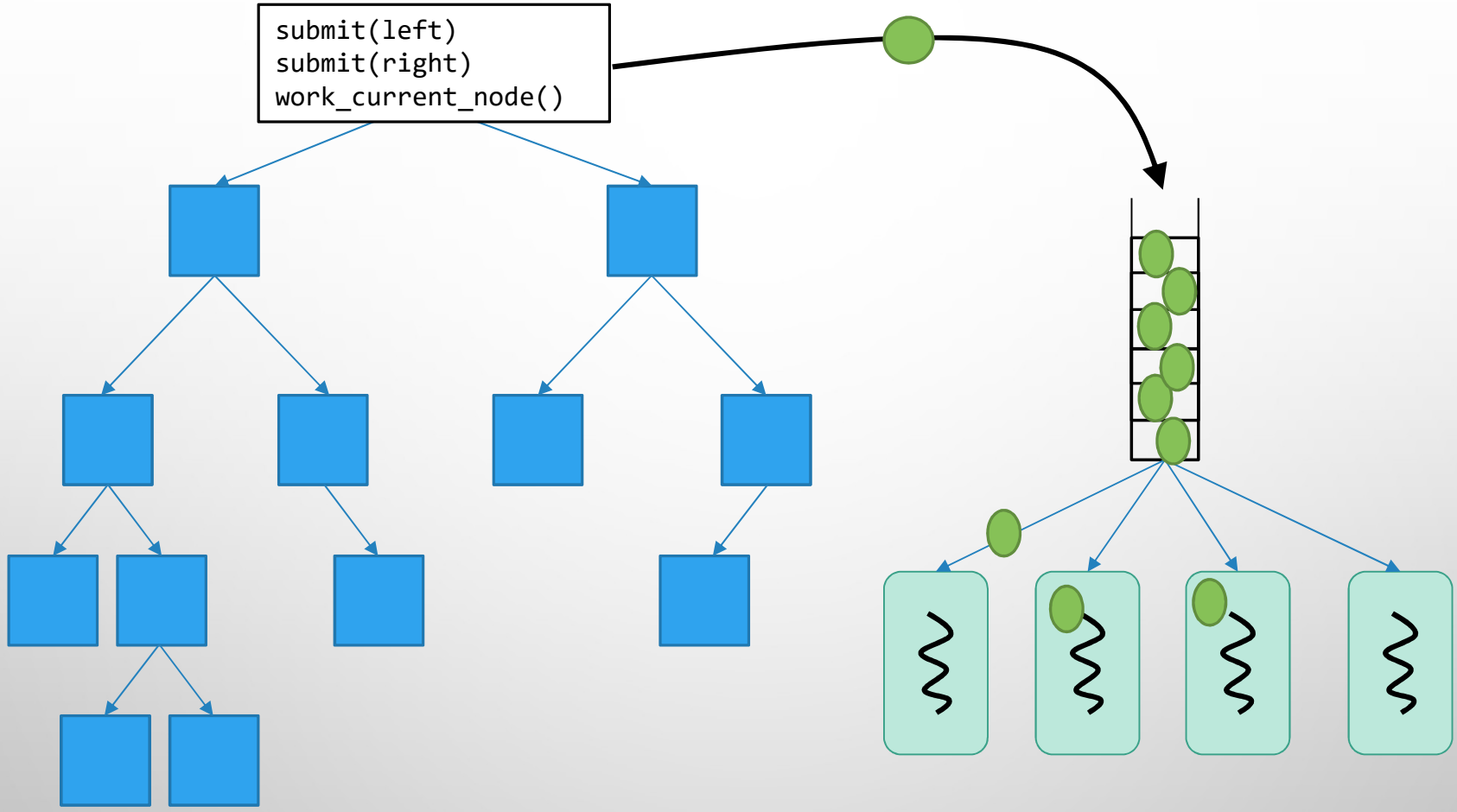
- Place chunks of work in a concurrent queue (we have done this before!)
- Place fixed number of threads in a “thread pool”
- Each thread gets works from the queue, pushes works in the queue

SOL2 – THREAD POOL

```
class thread_pool
{
	std::atomic<bool> done;
	thread_safe_queue<std::function<void()> > work_queue;
	void worker_thread()
	{
		while(!done) {
			std::function<void()> task;
			if(work_queue.try_pop(task)) {
				task();
			} else {
				std::this_thread::yield();
			}
		} //end while(done)
	} //end worker_thread
public:
	thread_pool()
	{
		…
		int thread_count
				= std::thread::hardware_concurrency();
		for(int t=0; t<thread_count ; t++) {
		{
			threads.push_back(std::thread(worker_thread, this));
		}
		…
	}
	void submit(std::function<void()> f)
	{
		work_queue.push(f)
	}
…
};
```

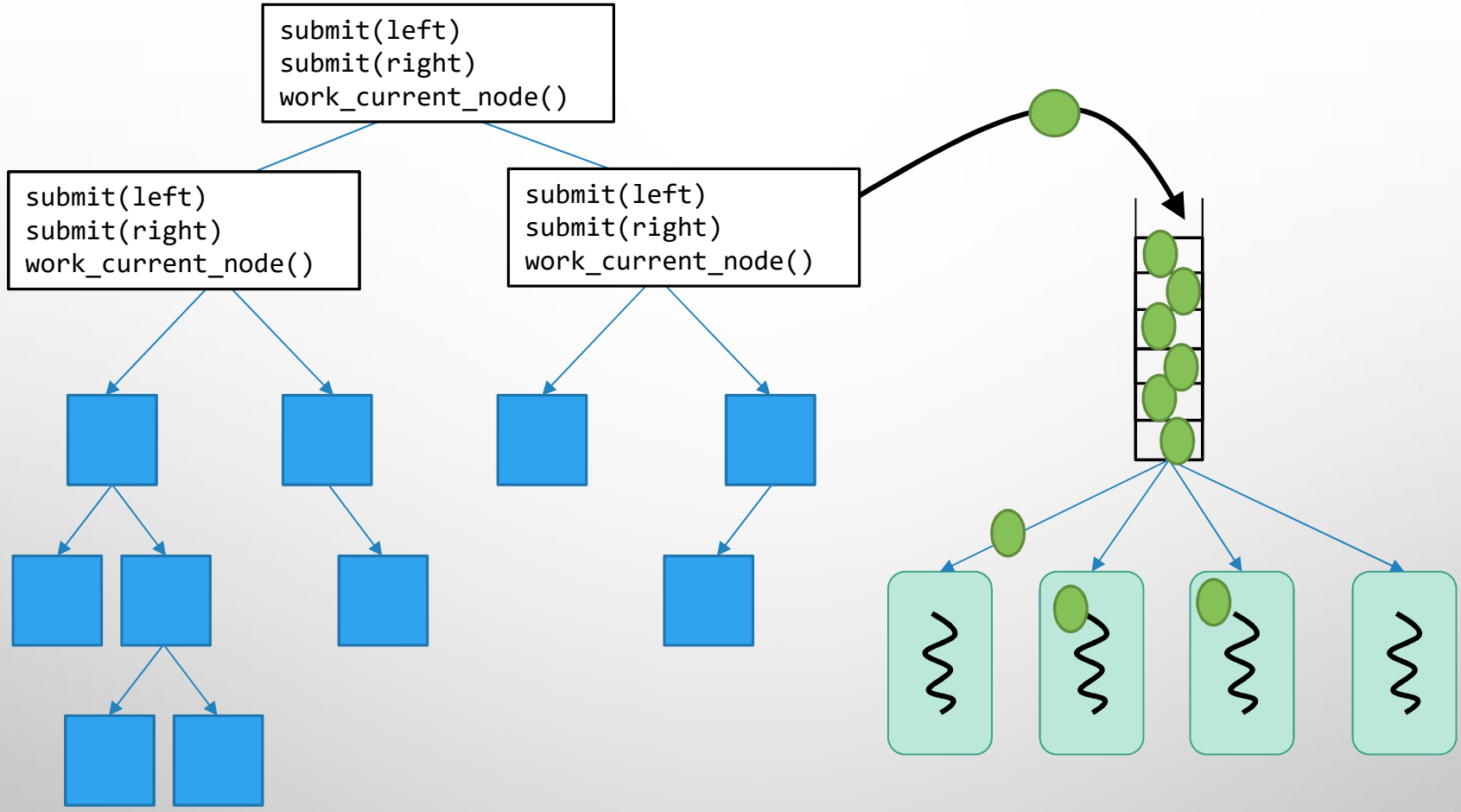
- Refer to section 4.6 of “Parallel Programming” textbook for a working example
  - or any other material

THREAD POOL (WORK QUEUE) - MOTIVATION



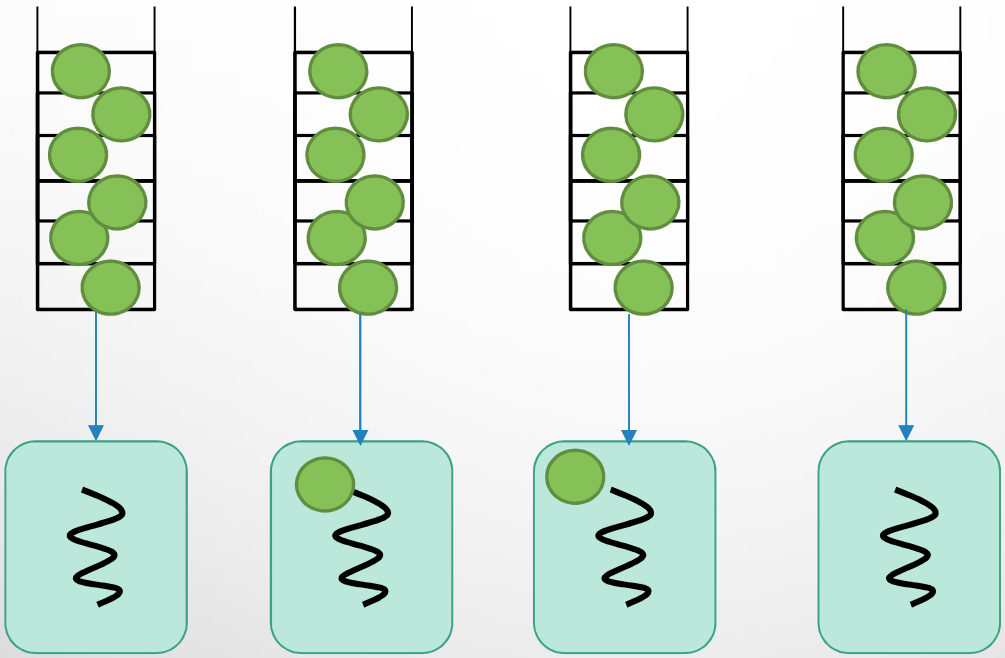
- Now you can work with 4 threads

THREAD POOL (WORK QUEUE) - MOTIVATION



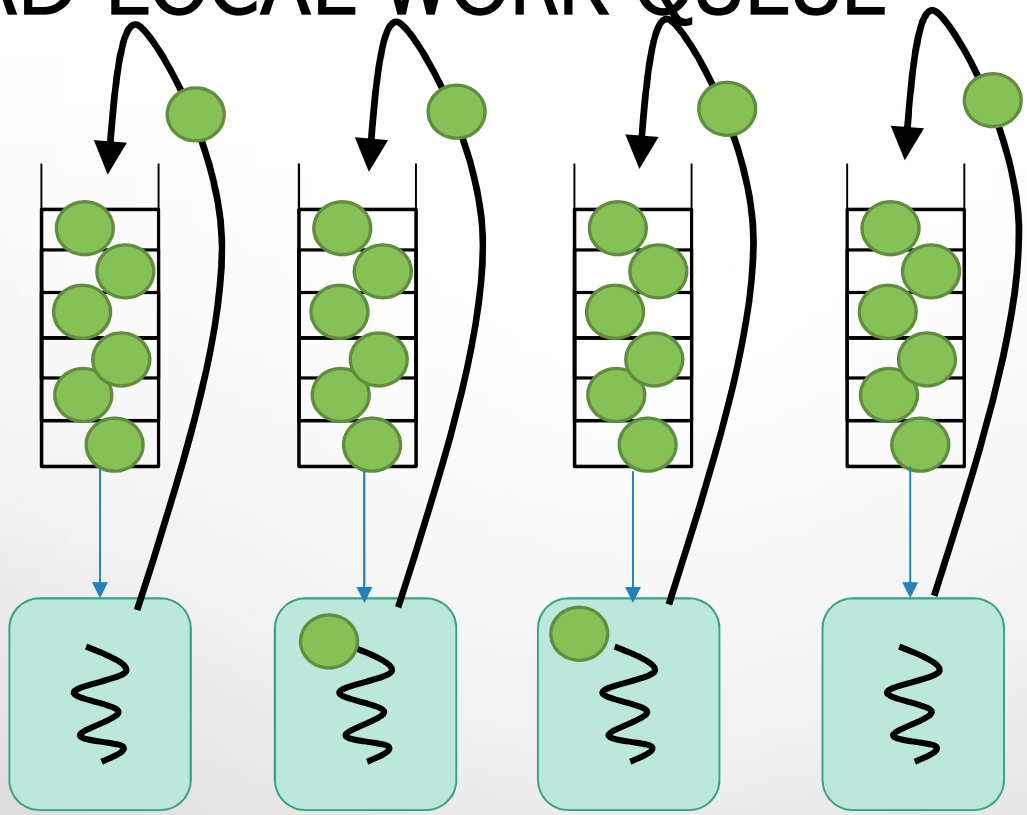
- Now you can work with 4 threads
- Any potential problem?

THREAD-LOCAL WORK QUEUE



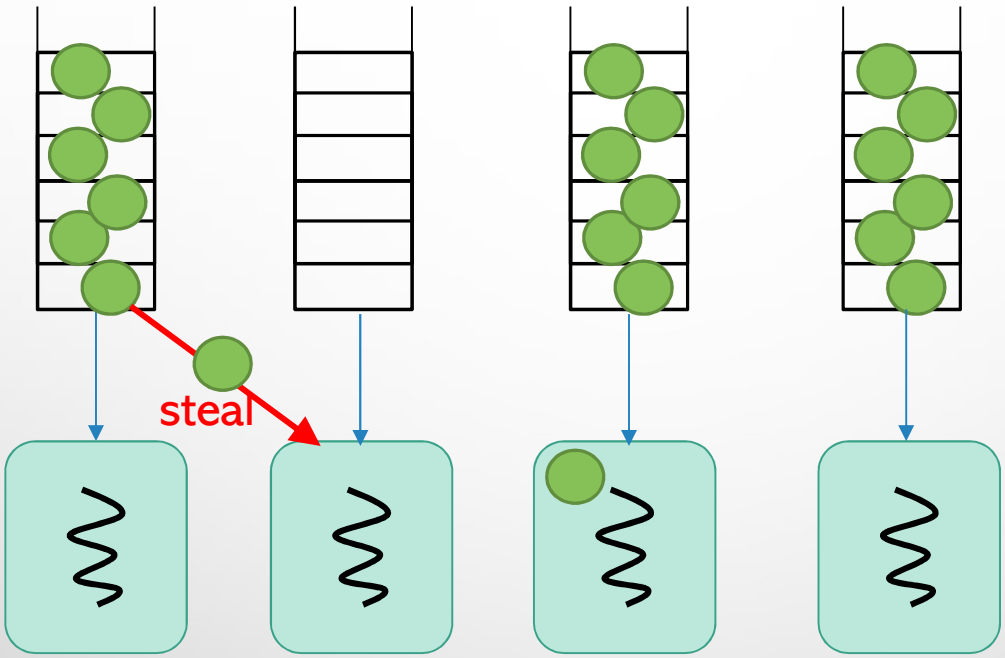
- Lock contention on the shared queue can become a problem
  - Everyone writes to the queue
  - Everyone reads from the queue
- Sol: use local queues!

THREAD-LOCAL WORK QUEUE



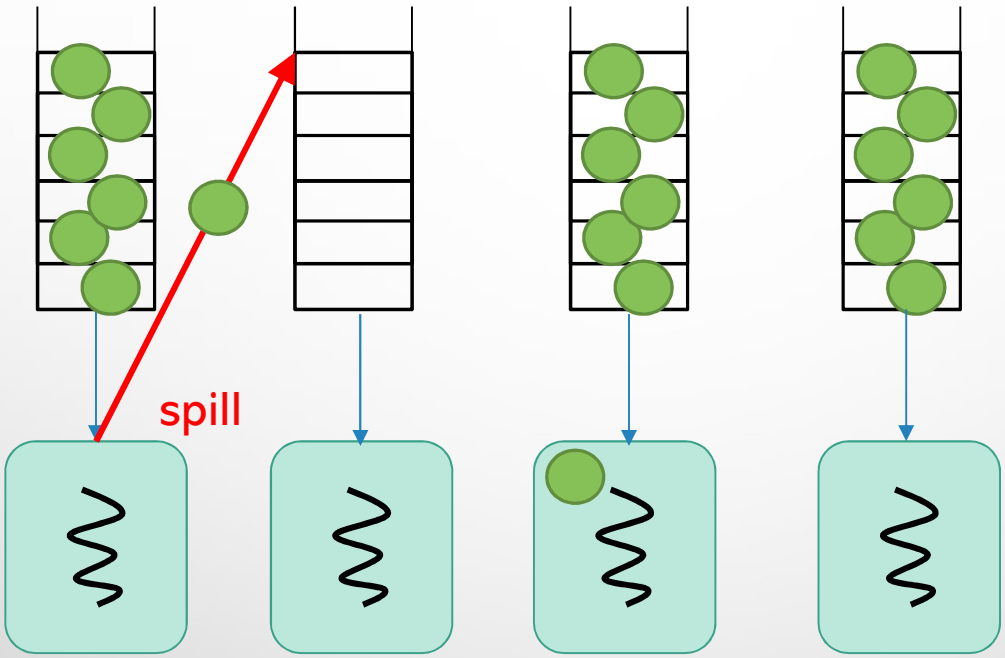
- Sol: use local queues!
- Only push to the own local queue
- Only pull from the own local queue
- == Static assignment???

THREAD-LOCAL WORK QUEUE



- Work stealing:
  - Stream from other queues when the local queue is empty
- All queues have
  - Single producer
  - Multiple consumer
  - This is easier to implement!

THREAD-LOCAL WORK QUEUE



- Work spilling:
  - Move items to other queues when the local queue is overloaded
- All queues have
  - Multiple producer
  - Single consumer

SPILLING VS STEALING

	Stealing	Spilling
Queue reader	Multiple	Single
Queue writer	Single	Multiple
Who pays the cost of moving the job	Idle thread	Overloaded thread
Good balance when	A few idle threads	A few overloaded threads